

Mockito

Mocks, stubs and behavior verification in Java tests.

What it is

Mockito is a Java library for creating mocks, stubbing behavior and verifying interactions in tests. It is commonly used with JUnit.

Official site and documentation

Use the official Mockito source as the primary reference: <https://site.mockito.org/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

When to use

Use it when a unit should be tested independently from dependencies such as gateways, clients, repositories or external services.

Add to project

Add Mockito through the project build manager (Maven/Gradle), using a compatible project-approved version. It is not a desktop tool installed in PATH.

Core concepts

Learn `mock()`, `@Mock`, `@InjectMocks`, `when(...).thenReturn(...)`, `verify(...)` and argument matchers. Avoid mocking everything.

QA Automation

Mockito is useful when QA works close to code, service/component tests or maintains Java suites with developers.

Common mistakes

Too many mocks make tests brittle and unrealistic. Test meaningful behavior and use real integration where risk requires it.

Installation and configuration - overview

Always use the official website and documentation. Check prerequisites and compatible versions, install for your operating system, configure environment variables or PATH only when required, validate the installation with the recommended command or screen, and run a first project/test before integrating it into the QA workflow.

How to start safely

Start from the official Mockito site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Mockito, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Mockito fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Mockito still delivers value as the product evolves.

Common issues and troubleshooting

When Mockito fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.